DOCUMENTAÇÃO TÉCNICA

BANCO DE DADOS FÍSICO — PETSHOP

Projeto acadêmico | Engenharia de Software

Instituição	Universidade Cidade de São Paulo — UNICID
Disciplina	Banco de Dados
Professor	Clovis Jose Ramos Ferraro
Integrantes	Maria Fernanda Martins Vaz; João Pedro Maia Amaral Monteiro; Rene Silveira Luz De Aguiar; Igor Souza do Nascimento; José Vitor da Silva Freitas
Responsabilidade documentada	Implementação e validação do banco físico
Data de referência informada	05/11/2026

Documento elaborado a partir do backup final exportado do ambiente MySQL Aiven e das validações registradas durante o projeto.

1. Objetivo e escopo

Este documento descreve a estrutura física efetivamente presente no backup final do projeto PetShop: tabelas, colunas, chaves, restrições de integridade, gatilhos (triggers) e dados de demonstração. O backup analisado identifica o banco **PetShop** e registra a versão do servidor **MySQL 8.4.11**.

Fonte de verdade: arquivo petshop_backup-final.sql fornecido para esta consolidação. A documentação descreve o conteúdo desse arquivo; alterações posteriores no servidor não estarão refletidas até que um novo backup seja gerado.

2. Resumo do inventário físico

Elemento	Quantidade identificada
Tabelas	27
Chaves primárias	27
Chaves estrangeiras (FK)	40
Restrições CHECK	27
Índices/constraints UNIQUE	15
Triggers	18
Views	0

Todas as tabelas identificadas utilizam InnoDB. O backup contém comandos de criação das tabelas, dados e definições de triggers. Não foi encontrada instrução CREATE VIEW no arquivo.

3. Inventário de tabelas e colunas

A relação abaixo resume os nomes das colunas e seus tipos SQL principais. Detalhes completos de nulabilidade, valores padrão, índices e expressões estão preservados no próprio DDL do backup.

agendamento

id_agendamento — bigint
id_cliente — bigint
id_animal — bigint
id_funcionario — bigint
data_hora_agendada — timestamp
status — varchar(20)
motivo_cancelamento — varchar(255)
observacoes — text
data_registro — timestamp

agendamento_servico

id_agendamento — bigint
id_servico — bigint

animal

id_animal — bigint
id_cliente — bigint
nome — varchar(80)
especie — varchar(40)
raca — varchar(60)
porte — varchar(20)
data_nascimento — date
vacinacao_em_dia — tinyint(1)
observacoes_saude — text
ativo — tinyint(1)

atendimento

id_atendimento — bigint
id_agendamento — bigint
id_funcionario — bigint
data_hora_inicio — timestamp
data_hora_fim — timestamp
status — varchar(20)
observacoes — text

atendimento_servico

id_atendimento — bigint
id_servico — bigint
quantidade — int
valor_cobrado — decimal(10,2)

baixa_lote

id_item_venda — bigint
id_lote — bigint
quantidade_baixada — int
data_hora_baixa — timestamp

cliente

id_pessoa — bigint
data_cadastro — date
ativo — tinyint(1)

compra

id_compra — bigint
id_fornecedor — bigint
id_funcionario — bigint
data_compra — date
status — varchar(20)
numero_nota_fiscal — varchar(30)
observacoes — text

conta_pagar

id_conta_pagar — bigint
id_compra — bigint
numero_parcela — int
valor — decimal(12,2)
data_emissao — date
data_vencimento — date
status — varchar(20)

conta_receber

id_conta_receber — bigint
id_venda — bigint
numero_parcela — int
valor — decimal(12,2)
data_emissao — date
data_vencimento — date
status — varchar(20)

fornecedor

id_fornecedor — bigint
razao_social — varchar(150)
nome_fantasia — varchar(150)
cnpj — char(14)
email — varchar(150)
end_logradouro — varchar(150)
end_numero — varchar(10)
end_complemento — varchar(60)
end_bairro — varchar(80)
end_cidade — varchar(80)
end_uf — char(2)
end_cep — char(8)
ativo — tinyint(1)

fornecedor_telefone

id_fornecedor — bigint
telefone — varchar(20)

funcionario

id_pessoa — bigint
id_perfil — bigint
cargo — varchar(60)
data_admissao — date
ativo — tinyint(1)

item_compra

id_item_compra — bigint
id_compra — bigint
id_produto — bigint
quantidade_comprada — int
custo_unitario — decimal(10,2)
subtotal — decimal(12,2)

item_venda

id_item_venda — bigint
id_venda — bigint
id_produto — bigint
quantidade — int
preco_unitario — decimal(10,2)
desconto_item — decimal(10,2)
subtotal — decimal(12,2)

lote

id_lote — bigint
id_item_compra — bigint
codigo_lote — varchar(40)
data_entrada — date
data_validade — date
quantidade_recebida — int

movimento_caixa

id_movimento — bigint
id_funcionario — bigint
id_recebimento — bigint
id_pagamento — bigint
data_hora — timestamp
tipo — varchar(10)
valor — decimal(12,2)
descricao — varchar(255)

pagamento

id_pagamento — bigint
id_conta_pagar — bigint
id_funcionario — bigint
data_hora — timestamp
valor_pago — decimal(12,2)
forma_pagamento — varchar(30)

perfil_acesso

id_perfil — bigint
nome — varchar(60)
descricao — varchar(255)

pessoa

id_pessoa — bigint
nome — varchar(120)
cpf — char(11)
email — varchar(150)
end_logradouro — varchar(150)
end_numero — varchar(10)
end_complemento — varchar(60)
end_bairro — varchar(80)
end_cidade — varchar(80)
end_uf — char(2)
end_cep — char(8)

pessoa_telefone

id_pessoa — bigint
telefone — varchar(20)

produto

id_produto — bigint
nome — varchar(120)
descricao — varchar(255)
categoria — varchar(60)
unidade_medida — varchar(10)
preco_venda — decimal(10,2)
estoque_minimo — int
ativo — tinyint(1)

produto_fornecedor

id_fornecedor — bigint
id_produto — bigint

recebimento

id_recebimento — bigint
id_conta_receber — bigint
id_funcionario — bigint
data_hora — timestamp
valor_recebido — decimal(12,2)
forma_pagamento — varchar(30)

servico

id_servico — bigint
nome — varchar(80)
descricao — varchar(255)
preco_base — decimal(10,2)
duracao_estimada_min — int
ativo — tinyint(1)

transporte

id_transporte — bigint
id_atendimento — bigint
id_funcionario — bigint
tipo — varchar(30)
origem_logradouro — varchar(150)
origem_numero — varchar(10)
origem_complemento — varchar(60)
origem_bairro — varchar(80)
origem_cidade — varchar(80)
origem_uf — char(2)
origem_cep — char(8)
destino_logradouro — varchar(150)
destino_numero — varchar(10)
destino_complemento — varchar(60)
destino_bairro — varchar(80)
destino_cidade — varchar(80)
destino_uf — char(2)
destino_cep — char(8)
data_hora_prevista — timestamp
valor — decimal(10,2)
status — varchar(20)

venda

id_venda — bigint
id_cliente — bigint
id_funcionario — bigint
id_atendimento — bigint
data_hora_venda — timestamp
status — varchar(20)
desconto_total — decimal(10,2)
observacoes — text

4. Relacionamentos — chaves estrangeiras

Foram identificadas 40 declarações FOREIGN KEY no DDL. A tabela abaixo apresenta a tabela de origem, coluna(s), tabela referenciada e coluna(s) referenciada(s).

Origem	Constraint	Coluna(s) de origem	Referência
agendamento	agendamento_ibfk_1	id_animal, id_cliente	animal (id_animal, id_cliente)
agendamento	agendamento_ibfk_2	id_funcionario	funcionario (id_pessoa)
agendamento	agendamento_ibfk_3	id_cliente	cliente (id_pessoa)
agendamento_servico	agendamento_servico_ibfk_1	id_agendamento	agendamento (id_agendamento)
agendamento_servico	agendamento_servico_ibfk_2	id_servico	servico (id_servico)
animal	animal_ibfk_1	id_cliente	cliente (id_pessoa)
atendimento	atendimento_ibfk_1	id_agendamento	agendamento (id_agendamento)
atendimento	atendimento_ibfk_2	id_funcionario	funcionario (id_pessoa)
atendimento_servico	atendimento_servico_ibfk_1	id_atendimento	atendimento (id_atendimento)
atendimento_servico	atendimento_servico_ibfk_2	id_servico	servico (id_servico)
baixa_lote	baixa_lote_ibfk_1	id_item_venda	item_venda (id_item_venda)
baixa_lote	baixa_lote_ibfk_2	id_lote	lote (id_lote)
cliente	cliente_ibfk_1	id_pessoa	pessoa (id_pessoa)
compra	compra_ibfk_1	id_fornecedor	fornecedor (id_fornecedor)
compra	compra_ibfk_2	id_funcionario	funcionario (id_pessoa)
conta_pagar	conta_pagar_ibfk_1	id_compra	compra (id_compra)
conta_receber	conta_receber_ibfk_1	id_venda	venda (id_venda)
fornecedor_telefone	fornecedor_telefone_ibfk_1	id_fornecedor	fornecedor (id_fornecedor)

Origem	Constraint	Coluna(s) de origem	Referência
funcionario	funcionario_ibfk_1	id_pessoa	pessoa (id_pessoa)
funcionario	funcionario_ibfk_2	id_perfil	perfil_acesso (id_perfil)
item_compra	item_compra_ibfk_1	id_compra	compra (id_compra)
item_compra	item_compra_ibfk_2	id_produto	produto (id_produto)
item_venda	item_venda_ibfk_1	id_venda	venda (id_venda)
item_venda	item_venda_ibfk_2	id_produto	produto (id_produto)
lote	lote_ibfk_1	id_item_compra	item_compra (id_item_compra)
movimento_caixa	movimento_caixa_ibfk_1	id_funcionario	funcionario (id_pessoa)
movimento_caixa	movimento_caixa_ibfk_2	id_recebimento	recebimento (id_recebimento)
movimento_caixa	movimento_caixa_ibfk_3	id_pagamento	pagamento (id_pagamento)
pagamento	fk_pagamento_funcionario	id_funcionario	funcionario (id_pessoa)
pagamento	pagamento_ibfk_1	id_conta_pagar	conta_pagar (id_conta_pagar)
pessoa_telefone	pessoa_telefone_ibfk_1	id_pessoa	pessoa (id_pessoa)
produto_fornecedor	produto_fornecedor_ibfk_1	id_fornecedor	fornecedor (id_fornecedor)
produto_fornecedor	produto_fornecedor_ibfk_2	id_produto	produto (id_produto)
recebimento	fk_recebimento_funcionario	id_funcionario	funcionario (id_pessoa)
recebimento	recebimento_ibfk_1	id_conta_receber	conta_receber (id_conta_receber)
transporte	transporte_ibfk_1	id_atendimento	atendimento (id_atendimento)

Origem	Constraint	Coluna(s) de origem	Referência
transporte	transporte_ibfk_2	id_funcionario	funcionario (id_pessoa)
venda	venda_ibfk_1	id_cliente	cliente (id_pessoa)

Origem	Constraint	Coluna(s) de origem	Referência
venda	venda_ibfk_2	id_funcionario	funcionario (id_pessoa)
venda	venda_ibfk_3	id_atendimento	atendimento (id_atendimento)

5. Restrições CHECK

Foram identificadas 27 constraints CHECK no DDL. Elas complementam as chaves e validam condições de domínio, valores positivos, datas e consistência entre campos.

atendimento.chk_atendimento_periodo
((`data_hora_fim` is null) or (`data_hora_fim` >= `data_hora_inicio`))

atendimento_servico.chk_atendimento_servico_quantidade
(`quantidade` > 0)

atendimento_servico.chk_atendimento_servico_valor
(`valor_cobrado` >= 0)

baixa_lote.chk_baixa_lote_quantidade
(`quantidade_baixada` > 0)

conta_pagar.chk_conta_pagar_datas
(`data_vencimento` >= `data_emissao`)

conta_pagar.chk_conta_pagar_parcela
(`numero_parcela` > 0)

conta_pagar.chk_conta_pagar_valor
(`valor` > 0)

conta_receber.chk_conta_receber_datas
(`data_vencimento` >= `data_emissao`)

conta_receber.chk_conta_receber_parcela
(`numero_parcela` > 0)

conta_receber.chk_conta_receber_valor
(`valor` > 0)

item_compra.chk_item_compra_custo
(`custo_unitario` >= 0)

item_compra.chk_item_compra_quantidade
(`quantidade_comprada` > 0)

item_venda.chk_item_venda_desconto
((`desconto_item` >= 0) and (`desconto_item` <= (`quantidade` * `preco_unitario`)))

item_venda.chk_item_venda_preco
(`preco_unitario` >= 0)

item_venda.chk_item_venda_quantidade
(`quantidade` > 0)

lote.chk_lote_data_validade
((`data_validade` is null) or (`data_validade` >= `data_entrada`))

lote.chk_lote_quantidade_recebida
(`quantidade_recebida` > 0)

movimento_caixa.chk_movimento_origem_tipo
(((`id_recebimento` is not null) and (`id_pagamento` is null) and (`tipo` = _utf8mb4'ENTRADA')) or ((`id_recebimento` is null) and (`id_pagamento` is not null) and (`tipo` = _utf8mb4'SAIDA')))

movimento_caixa.chk_movimento_valor_positivo
(`valor` > 0)

pagamento.chk_pagamento_valor
(`valor_pago` > 0)

produto.chk_produto_estoque_minimo
(`estoque_minimo` >= 0)

produto.chk_produto_preco_venda
(`preco_venda` >= 0)

recebimento.chk_recebimento_valor
(`valor_recebido` > 0)

servico.chk_servico_duracao
((`duracao_estimada_min` is null) or (`duracao_estimada_min` > 0))

servico.chk_servico_preco_base
(`preco_base` >= 0)

transporte.chk_transporte_valor
(`valor` >= 0)

venda.chk_venda_desconto_total
(`desconto_total` >= 0)

6. Restrições UNIQUE

Foram identificados 15 índices/restrições UNIQUE. Abaixo estão os nomes e as colunas protegidas conforme o DDL.

animal.animal_index_0 — (id_animal,id_cliente)

atendimento.id_agendamento — (id_agendamento)

compra.compra_index_1 — (id_fornecedor,numero_nota_fiscal)

conta_pagar.conta_pagar_index_6 — (id_compra,numero_parcela)

conta_receber.conta_receber_index_5 — (id_venda,numero_parcela)

fornecedor.cnpj — (cnpj)

item_compra.item_compra_index_2 — (id_compra,id_produto)

item_venda.item_venda_index_4 — (id_venda,id_produto)

lote.lote_index_3 — (id_item_compra,codigo_lote)

movimento_caixa.id_recebimento — (id_recebimento)

movimento_caixa.id_pagamento — (id_pagamento)

perfil_acesso.nome — (nome)

pessoa.cpf — (cpf)

transporte.id_atendimento — (id_atendimento)

venda.id_atendimento — (id_atendimento)

7. Triggers implementadas

O backup contém 18 triggers. A descrição funcional abaixo foi resumida a partir dos corpos SQL exportados.

trg_baixa_lote_before_insert — BEFORE INSERT em baixa_lote

Antes de inserir uma baixa, valida a existência do item de venda, do lote e do item de compra; confere se o produto do lote corresponde ao produto vendido; impede ultrapassar a quantidade recebida no lote ou a quantidade vendida.

trg_baixa_lote_before_update — BEFORE UPDATE em baixa_lote

Antes de atualizar uma baixa, impede trocar os vínculos de item de venda/lote e revalida os limites de quantidade e compatibilidade do produto.

trg_conta_pagar_before_update — BEFORE UPDATE em conta_pagar

Impede reduzir o valor da conta abaixo do total já pago e impede alterar compra/parcela quando já existem pagamentos vinculados.

trg_conta_receber_before_update — BEFORE UPDATE em conta_receber

Impede reduzir o valor da conta abaixo do total já recebido e impede alterar venda/parcela quando já existem recebimentos vinculados.

trg_item_compra_before_update — BEFORE UPDATE em item_compra

Protege o identificador e os dados históricos do item de compra após criação de lotes; impede reduzir a quantidade comprada abaixo do total recebido.

trg_item_venda_before_update — BEFORE UPDATE em item_venda

Protege o identificador e os dados comerciais do item de venda após baixas; impede reduzir a quantidade vendida abaixo do total baixado.

trg_lote_before_insert — BEFORE INSERT em lote

Valida o item de compra e impede que a soma das quantidades recebidas em lotes ultrapasse a quantidade comprada.

trg_lote_before_update — BEFORE UPDATE em lote

Impede transferir lote para outro item de compra; valida quantidade comprada e impede reduzir o lote abaixo do que já foi baixado ou ultrapassar o total comprado.

trg_pagamento_before_insert — BEFORE INSERT em pagamento

Valida a existência da conta a pagar e impede que a soma dos pagamentos ultrapasse o valor da conta.

trg_pagamento_after_insert — AFTER INSERT em pagamento

Cria automaticamente um movimento de caixa do tipo SAIDA vinculado ao pagamento.

trg_pagamento_before_update — BEFORE UPDATE em pagamento

Protege o identificador e a conta vinculada; valida limites financeiros e impede atualização incompatível com o histórico.

trg_pagamento_after_update — AFTER UPDATE em pagamento

Atualiza o movimento de caixa associado para refletir funcionário, data, valor e descrição do pagamento.

trg_pagamento_before_delete — BEFORE DELETE em pagamento

Remove o movimento de caixa associado antes da exclusão do pagamento.

trg_recebimento_before_insert — BEFORE INSERT em recebimento

Valida a existência da conta a receber e impede que a soma dos recebimentos ultrapasse o valor da conta.

trg_recebimento_after_insert — AFTER INSERT em recebimento

Cria automaticamente um movimento de caixa do tipo ENTRADA vinculado ao recebimento.

trg_recebimento_before_update — BEFORE UPDATE em recebimento

Protege o identificador e a conta vinculada; valida limites financeiros e impede atualização incompatível com o histórico.

trg_recebimento_after_update — AFTER UPDATE em recebimento

Atualiza o movimento de caixa associado para refletir funcionário, data, valor e descrição do recebimento.

trg_recebimento_before_delete — BEFORE DELETE em recebimento

Remove o movimento de caixa associado antes da exclusão do recebimento.

8. Regras de negócio protegidas no banco

Estoque e rastreabilidade: o conjunto de triggers de lote, item_compra, item_venda e baixa_lote protege a relação entre compra, recebimento por lote, venda e baixa de estoque. As validações cobrem compatibilidade de produto e limites quantitativos.

Financeiro e caixa: as triggers de pagamento e recebimento verificam limites em relação às contas vinculadas e mantêm movimentos de caixa correspondentes em operações de inclusão, atualização e exclusão. As contas a pagar e a receber também possuem proteção contra redução de valor abaixo do total já movimentado.

Restrições declarativas: PK, FK, UNIQUE e CHECK atuam em conjunto com as triggers. As regras de trigger não substituem as restrições declarativas; são camadas complementares.

9. Dados presentes no backup

O backup contém INSERTs para as 27 tabelas. A massa de dados é de demonstração/teste e inclui pessoas, clientes, funcionários, animais, agendamentos, serviços, compras, produtos, lotes, vendas, contas financeiras, recebimentos, pagamentos e movimentos de caixa.

Os dados incluem registros explicitamente identificados como temporários/de teste (por exemplo, observações de venda e atendimento). Por isso, devem ser apresentados como massa de dados acadêmica, e não como cadastro operacional real.

10. Consultas e evidências de validação

Durante a validação do projeto foram coletadas 17 evidências visuais: oito relacionadas a testes negativos de integridade do estoque; cinco relacionadas ao fluxo financeiro e movimentos de caixa; e quatro consultas demonstrativas de clientes/animais, agendamentos/serviços, estoque e vendas. O inventário de triggers também foi salvo na pasta de auditoria.

Os testes documentados demonstram cenários específicos. Não se deve interpretar a existência de 40 FKs ou 27 CHECKs como prova de que cada constraint foi individualmente submetida a um teste negativo. Da mesma forma, o backup comprova a definição das triggers, enquanto os prints comprovam apenas os cenários efetivamente executados.

11. Limitações e pontos de atenção

• Não existem views no backup analisado. Portanto, nenhuma view é declarada como implementada.
• O status das contas a pagar/receber presente nos dados não deve ser descrito como automaticamente quitado/conciliado apenas pela existência de pagamento ou recebimento. Os registros exportados mantêm status PENDENTE em contas consultadas durante a validação.
• O backup representa o estado exportado em 05/11/2026, conforme dados e timestamps nele contidos. Se o banco foi alterado depois da exportação, é necessária nova extração para atualizar esta documentação.
• A documentação não declara conformidade integral com 1FN, 2FN ou 3FN sem uma análise formal das dependências funcionais e das decisões de modelagem.

12. Conclusão

O backup final analisado registra 27 tabelas InnoDB, 40 chaves estrangeiras, 27 constraints CHECK, 15 restrições UNIQUE e 18 triggers, sem views. A estrutura inclui módulos de cadastro, atendimento, estoque, compras, vendas, financeiro e caixa. As regras de integridade implementadas concentram-se especialmente na rastreabilidade de estoque e na consistência dos movimentos financeiros.

Este documento deve ser entregue em conjunto com o backup SQL e as evidências visuais coletadas, mantendo os três artefatos como registros complementares: estrutura executável, explicação técnica e comprovação dos testes realizados.